

Web.Devs Team5

Building and Securing a REST API

1. Introduction to API Security

API is simply an entry point into any software application. It receives requests from other software applications and web sites and returns some data. When there is no one watching the gate of the API, everyone who discovers its address can access, modify or even delete the data. For our project the data are contained in the backup of a mobile money system's SMS messages (modified_sms_v2.xml), containing 1,691 messages. Each message includes amounts, balance, name and partial phone numbers, hence money-related data.

To complete the task we designed and implemented a REST API for the data in plain Python without using any framework such as Flask. First, the XML file was transformed into JSON objects. We created a dictionary structure for each SMS with 14 attributes including id, transaction_type, amount, fee, new_balance, sender, receiver, transaction_time. The process was difficult due to the fact that the XML contains only the general SMS information (address, date, body). The actual information about transactions is written within the body of the SMS. We used specific keywords and regular expressions to identify the type of the transaction and the values.

HTTP Basic authentication was used to make sure that our API is protected. All endpoints first check whether the Authorization header is present. In case it is not or if it contains incorrect credentials, 401 Unauthorized is returned by the server and nothing else. Proper handling of other cases was also implemented: 400 Bad Request if the body is missing or if it is not a proper JSON body, and 404 Not Found if there is an id or route that doesn't exist. Security-wise, it's also very important to prevent crashes on bad data.

2. Endpoints documentation

Server is running at <http://localhost:8000> (can be started by running `python3 api/server.py`). All endpoints should be authorized by Basic Auth. Demo user used in tests was admin with the password momo2026.

Method	Endpoint	What it does	Success code	Error codes
GET	/transactions	Lists all transactions	200	401
GET	/transactions/{id}	Shows one transaction	200	401, 404
POST	/transactions	Adds a new transaction	201	400, 401
PUT	/transactions/{id}	Updates an existing transaction	200	401, 404

DELETE	/transactions/{id}	Deletes a transaction	200	401, 404
--------	--------------------	-----------------------	-----	----------

GET /transactions

Returns all transactions (there are 1,691 in the beginning). It uses the same curl command as for GET without the id parameter and returns JSON array of objects as shown below.

```
curl -u admin:momo2026 http://localhost:8000/transactions
```

GET /transactions/{id}

Returns one transaction. Returns 404 when the id does not exist.

```
curl -u admin:momo2026 http://localhost:8000/transactions/1
```

Response 200 OK

```
{
  "id": 1,
  "transaction_type": "received",
  "amount": 2000.0,
  "sender": "Jane Smith",
  "new_balance": 2000.0,
  "transaction_time": "2024-05-10 16:30:51",
  ...
}
```

POST /transactions

Creates a new transaction. The body should contain transaction_type and amount. The server assigns an id to the new entity, therefore no id is supplied by the client.

```
curl -u admin:momo2026 -X POST http://localhost:8000/transactions \
-H "Content-Type: application/json" \
-d '{"transaction_type": "payment", "amount": 5000, "receiver": "Test Vendor"}'
```

Response 201 Created

```
{ "transaction_type": "payment", "amount": 5000,
  "receiver": "Test Vendor", "id": 1692 }
```

Response 400 Bad Request (a required field is missing)

```
{ "error": "Request body must include at least 'transaction_type' and 'amount'.",
  "status": 400 }
```

PUT /transactions/{id}

Updates only the properties that are sent. Other properties remain unchanged as well as the id property.

```
curl -u admin:momo2026 -X PUT http://localhost:8000/transactions/1692 \
-H "Content-Type: application/json" -d '{"amount":7500}'
```

Response 200 OK

```
{ "transaction_type": "payment", "amount": 7500,
  "receiver": "Test Vendor", "id": 1692 }
```

DELETE /transactions/{id}

```
curl -u admin:momo2026 -X DELETE http://localhost:8000/transactions/1692
```

Response 200 OK

```
{ "message": "Transaction 1692 deleted." }
```

Response 404 Not Found (already deleted or never existed)

```
{ "error": "Transaction 1692 not found.", "status": 404 }
```

Unauthorized request

```
curl -u wronguser:wrongpass http://localhost:8000/transactions/1
```

Response 401 Unauthorized

```
{ "error": "Unauthorized", "status": 401,
  "message": "Valid Basic Auth credentials are required." }
```

Error codes

An important aspect to consider is that the data is stored in memory. The operations POST, PUT and DELETE are performed as long as the server is up and running, but the changes are not saved to the file, and therefore, when you restart the server, the data will be restored to its original state. All the test scree

Code	Meaning	When it happens
200 OK	Success	GET, PUT or DELETE worked
201 Created	Record created	POST worked
400 Bad Request	Bad input	Empty body, invalid JSON or missing required fields
401 Unauthorized	Login failed	No credentials or wrong credentials
404 Not Found	Not found	The id does not exist or the URL is wrong

3. DSA Integration

We looked into two different algorithms to locate the record by its ID. The first algorithm is called linear search , it scans the whole list looking for the record with the ID value. The other algorithm is dictionary search ,every record is stored in the dictionary with the ID as a key.

For each dataset we executed the search for the last record in the list 1,000 times and counted the overall time needed to do it. The last record is the worst case for linear search since it has to go through all records until it meets the required one. The code also ensures that both search algorithms give the same results for each ID before timing.

Results

Time in seconds for 1,000 searches:

Records	Linear search	Dictionary	Dictionary faster by
20	0.000881	0.000134	6.6x
100	0.008989	0.000188	47.8x

500	0.026579	0.000088	301.5x
1,691	0.184559	0.000151	1,219.7x

Screenshot: screenshots/dsa_results.png)

In all cases, the dictionary was quicker and the discrepancy increased as the amount of data increased. Linear search took 0.000881 seconds for 20 records and 0.184559 seconds for 1691 records.

The dictionary kept its time at about 0.0001 seconds consistently. The slight differences in time, such as 0.000188 seconds for 100 records, were due to the small amount of noise introduced when measuring small numbers.

These timings represent the worst case scenario for linear search since I was searching for the last record every time.

- Why is dictionary lookup faster?

The linear search complexity is $O(n)$. In the worst case scenario it may have to check all of the records, therefore having twice as many records would require twice the amount of work. A Python dictionary is a form of hash table. It hashes the key, which tells Python the location of the value without going through the rest of the records. It gives an average lookup time of $O(1)$, independent of the number of records. The drawback is the increased memory usage and one time initialization required for building the dictionary. We do that initialization one time when we start the server, thus `GET /transactions/{id}` requires the dictionary.

4. Reflection on Basic Auth Limitations

Basic Auth was simple to implement and satisfies the assignment needs, but during its implementation we found out that it is insecure for real application.

The password is not encrypted. Authorization header is a `base64(username:password)` string. Base64 is encoding, not encryption, hence any malicious user capturing the request can easily decode it and obtain the password. If the API works via plain HTTP and not via HTTPS, it is extremely easy to intercept.

Password is sent with every request. Hence there are many opportunities for a potential attacker to intercept it, and it could be logged or cached by a proxy server. The client will also need to store the actual password.

No expiry and no logout. Once the credentials are stolen, they remain valid until the password change. It is impossible to revoke access only for certain devices.

No roles and permissions. Basic Auth verifies your identity, nothing else. In our API both users can perform any operations, including record deletion, which is probably not needed for a regular user.

No way to prevent password guessing. An attacker can continuously try different passwords, since we have neither rate limiting nor account lockout mechanisms, and there is no second step like MFA.

Our own code is vulnerable too. In `api/auth.py`, usernames and passwords are hardcoded in a dictionary in plain text. It is done just for demonstration purpose of the class. In practice, passwords must be stored as hashes (bcrypt), while secrets must be loaded from environment variables. Comparison should be performed using `hmac.compare_digest` function, so no information could be leaked via timing.

Alternative solutions

JWT (JSON Web Token). The user authenticates once using his username and password, and receives an expirable signed token. He sends this token in the header (Authorization: Bearer <token>) instead of password. Also, this token contains user's role, which allows us to set permissions on deleting. A problem with JWT is that it is hard to invalidate before its expiry.

OAuth 2.0. The application obtains limited access to the data via access token that is provided by the authorization server, and the application does not see the password of the user. It is more suitable for the case when a number of clients (mobile app, web app, third party companies) use the API.

Regardless of which approach we take, the API has to be protected with HTTPS to encrypt the tokens/passwords during transmission.

Conclusion

The MoMo XML file was parsed to JSON. All five CRUD endpoints were implemented behind Basic Auth and tested against linear search and dictionary look-up. The major conclusions are that dictionary look-up is more scalable than searching through the array and that Basic Auth is okay for educational purposes only. In case we decided to continue the project, we would implement HTTPS with JWT, hashed passwords, roles, and save our data in database.

Testing and Validation

The API was tested using curl and the project's Basic Auth credentials. The screenshot for each test can be seen in the screenshots directory.

Test	Request	Expected	Result	Screenshot
GET with valid credentials	GET /transactions/1	200 OK	200 OK, returned the record	01_get_success.png
GET with wrong password	GET /transactions/1	401 Unauthorized	401 Unauthorized, with a WWW-Authenticate header	02_unauthorized.png
POST a new transaction	POST /transactions	201 Created	201 Created, new record got id 1692	03_post_success.png

Test	Request	Expected	Result	Screenshot
PUT to update it	PUT /transactions/1692	200 OK	200 OK, amount changed to 2500 and receiver to "Updated User"	04_put_success.png
DELETE it	DELETE /transactions/1692	200 OK	200 OK, "Transaction 1692 deleted."	05_delete_success.png

All the five test scenarios generated the status codes that I had anticipated. The API uses a 401 to reject the wrong credentials, and the creation, updating, and deletion of a transaction was successful.

Screenshot: screenshots/dsa_results.png)

